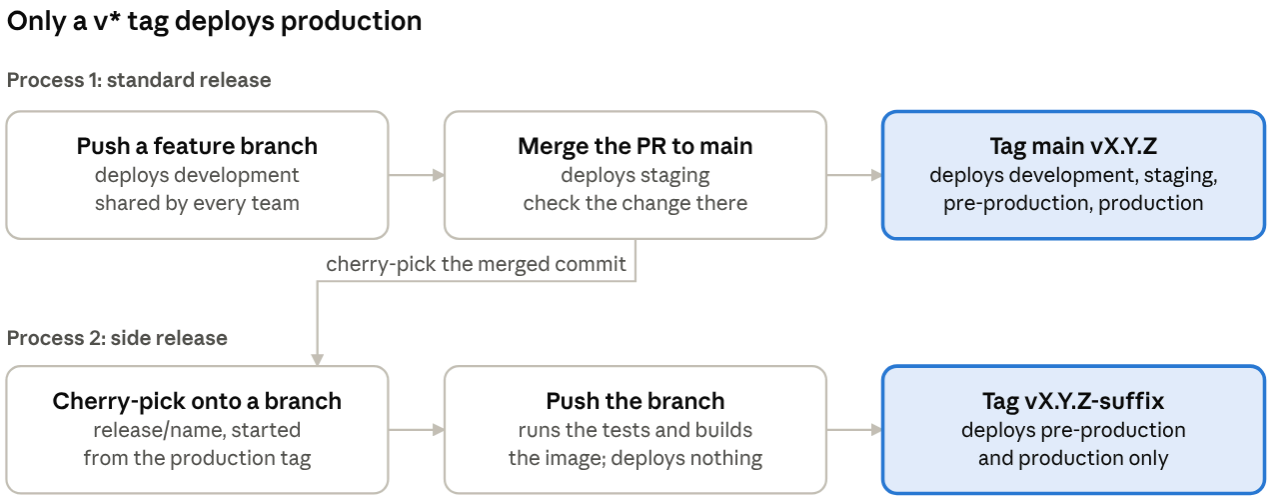
DIS Environments and Releases

Oct 29, 2026 · @Neptali Vasquez Baxman

DIS runs in four environments. Branch pushes and merges update development and staging automatically; production changes only when someone pushes a `v*` tag, through one of two processes: a **standard release** of everything on `main`, or a **side release** of one fix.

At a glance



the two ways to production

A merge to `main` stops at staging; only a tag reaches production. A side release copies one merged commit onto a branch cut from the production tag, so production gets that fix and nothing else from `main`.

The four environments

Each environment is an Azure Container App in resource group `grip-data-interop-service`, running the same image (`gripdatainteropacr.azurecr.io/dis-backend`) with its own database and settings.

Environment	Azure app	URL	What it is for
Development	`dis-dev`	https://dis-dev.yellowisland-7c2a284e.eastus2.azurecontainerapps.io	Trying a branch. Shared by every team: the last push wins.
Staging	`dis-staging`	https://dis-staging.alzheimersdata.org	Always runs `main`. Partners' test systems (ADWB stage, HDR UK preprod) use it.
Pre-production	`dis-pre-prod`	https://dis-pre-prod.yellowisland-7c2a284e.eastus2.azurecontainerapps.io	The last stop before production: every release tag deploys it.
Production	`dis-prod`	https://dis-prod.alzheimersdata.org	Live ADWB and live partners.

Every environment answers on `/api/dis/health-check` (healthy = `"status":"healthy"`) and shows its API on `/docs`.

How environments get updated

Nothing is deployed by hand. Every push to GitHub runs the pipeline (`.github/workflows/ci-cd.yml`), and the kind of push decides where it deploys.

You push	Development	Staging	Pre-production	Production
A feature branch	✓			
A merge to `main`		✓		
A `release/pre-<name>` branch	✓	✓	✓	
A release tag `vX.Y.Z`	✓	✓	✓	✓
A side-release branch `release/<name>`				

You push	Development	Staging	Pre-production	Production
A side-release tag `vX.Y.Z-<suffix>`			✓	✓

A side-release branch only runs the tests and builds the image. Side releases skip development and staging because those already run `main`, with newer database migrations.

What happens on each push. Step 1 runs once; steps 2 to 4 run for each environment the push targets, all in parallel:

1. Runs the backend tests, builds the image and pushes it to the registry, tagged with the commit SHA (branches) or the version (tags: `v1.1.12` → `1.1.12`).
2. Points the Container App at the new image, with that environment's settings.
3. Waits up to 10 minutes for the new revision to report Healthy. If it doesn't, the job fails and the previous revision keeps serving.
4. Calls `/api/dis/health-check`, then runs the Postman tests against that environment.

Database migrations run when the container starts (`alembic upgrade head` in `code/backend/entrypoint.sh`). A failing migration stops the container from starting, which step 3 catches.

Settings and secrets

- Settings come from `code/example.env` plus per-environment overrides in the pipeline (environment name, log level, workers, database). A change to `example.env` reaches every environment on its next deploy.
- Secrets live in one shared Key Vault, `grip-dis-keyvault-shared`. Most are read by their exact name, so one value serves every environment. API keys first try an environment-specific copy, `<NAME>-<environment>` (for example `-production`). A new secret must exist before the deploy that needs it.
- Provider settings live in the `data_providers` table, changed through an Alembic migration. DIS reads them the first time a provider is used after the container starts.

Process 1: standard release

The default way to production: tag `main`, and everything merged since the last release ships together. Use it whenever everything on `main` is cleared for production.

1. **Merge through a reviewed PR.** The merge deploys staging; check your change there.

2. **List what will ship**, as PR numbers on `main` that production doesn't have yet, and have each owning team confirm theirs (for the production tag, see "Which version is in production?" under "After a deploy"):

   ```
   PROD=<production tag>; comm -23 <(git log --format=%s $PROD..origin/main |
   grep -o '(#[0-9]*)$' | sort -u) <(git log --format=%s $PROD | grep -o '(#
   [0-9]*)$' | sort -u)
   ```

3. **Pick the next version**: one patch number above the highest tag, side releases included. `git fetch origin --tags` then `git tag --sort=-v:refname | head -3`.
4. **Tag `main` and push the tag.** This is the moment production deploys:

   ```
   git tag v1.1.12 origin/main
   git push origin v1.1.12
   ```

5. **Watch the run** in GitHub Actions ("DIS - CI/CD Pipeline" for the tag), or `gh run list --branch v1.1.12 --limit 1` then `gh run watch <run id>`. All four environments deploy at the same time.
6. **Check production** (see "After a deploy").

To try a release in pre-production before tagging, push a `release/pre-<name>` branch: it deploys development, staging and pre-production, but not production.

Process 2: side release

A side release ships one fix on top of what production already runs, leaving the rest of `main` behind. Use it only when `main` holds work that isn't cleared for production. For example, `v1.1.11-hdruk-only` (28 October 2026) was `v1.1.10-hdruk-only` plus PR #172, while `main` held the NACC provider.

The example commands below assume production runs `v1.1.12` and the fix is for NACC.

1. **Merge the fix to `main` first**, through a reviewed PR, and check it on staging. The side release only copies that commit. A fix that exists only on a release branch is lost at the next standard release.
2. **Find the production tag and the next version** (see "Which version is in production?"). The next version is one above the highest tag: here `v1.1.13`.
3. **Create a branch that starts at the production tag.** Your folder then holds production's code; `main` is untouched.

   ```
   git fetch origin --tags
   git switch -c release/v1.1.13-nacc-fix v1.1.12
   ```

4. **Copy the fix** with `git cherry-pick -x <commit>`, using the squash commit its PR left on `main` (its subject ends with the PR number). For several PRs, pick them one at a time, in the order they merged.
5. **On a conflict, stop and decide.** The fix edits code production doesn't have yet. Either cherry-pick the PR it depends on too, or keep production's side and drop only the parts that belong to that PR. Say in the commit message what you dropped.
6. **Check exactly what ships**: `git log --oneline v1.1.12..HEAD` must list only your picks, and `git diff --stat v1.1.12 HEAD` only the files you expect.
7. **Push the branch** with `git push -u origin release/v1.1.13-nacc-fix`. The pipeline runs the tests and builds the image; nothing deploys. Wait for it to pass.
8. **Tag the branch and push the tag.** Pre-production and production deploy at the same time:

   ```
   git tag v1.1.13-nacc-fix
   git push origin v1.1.13-nacc-fix
   ```

9. **Check production**, then record which PRs production now runs on top of the previous tag.

Naming. Any `v*` tag with a suffix after the version, and any branch under `release/` except `release/pre-*`, is a side release (PR #174). Tags made before #174 (`v1.1.11-hdruk-only` and older) still carry the old pipeline: a side release cut from one of them must cherry-pick #174's commit first, taking #174's version where it conflicts.

Limits.

- **No database migrations.** A new migration chains onto `main`'s newest one, which production doesn't have, so the container would fail to start. Changes with a file under `code/backend/alembic/versions/` go out in a standard release.
- **The version number is used up.** The next release, standard or side, takes the number after it.
- **Keep side releases rare.** Each one makes production drift further from `main`, and the next standard release has to bring it back in line.

After a deploy

A release is done when the run is green, production reports healthy, and the change works there.

- **The run is green**, including "Verify new revision is healthy" and the Postman tests for each environment it deployed.

- **Production is healthy**: https://dis-prod.alzheimersdata.org/api/dis/health-check returns `"status":"healthy"`, with the database and key management healthy.
- **The change works**: check the behaviour itself, not just the health check.
- **Logs**, if needed: `az containerapp logs show --name dis-prod --resource-group grip-data-interop-service --tail 50`.

Which version is in production?

The last `v*` tag whose pipeline run succeeded:

```
gh run list --workflow ci-cd.yml --limit 100 --json headBranch,conclusion -q
'[.[] | select(.headBranch | test("^v[0-9]")) | select(.conclusion ==
"success")][0].headBranch'
```

Rolling back

Situation	What to do
The new revision never became healthy (the run fails at "Verify new revision is healthy")	Nothing changed for users: the previous revision keeps serving. Read the logs the job printed, fix on `main`, release again.
Healthy but wrong, and the release added no migrations	Redeploy the previous version: re-run "Deploy to Production" in the previous tag's pipeline run (GitHub allows re-runs for 30 days), or `az containerapp update --name dis-prod --resource-group grip-data-interop-service --image gripdatainteropacr.azurecr.io/dis-backend:<previous version>`. Then fix forward.
Healthy but wrong, and the release added migrations	Don't redeploy the old image: its code can't start against the newer database schema. Revert the change on `main` and release forward, with a migration that undoes the schema change if needed.

Rules of thumb

- **Keep `main` releasable.** Only merge what can go to production, or keep it switched off there. Work on `main` that isn't cleared forces everyone into side releases.

- **Anyone with write access can deploy production.** No approval step guards a `v*` tag, so agree who tags and announce each release.
- **Versions only go up, and pushed tags never move.** Fix a bad release with a new one.
- **Secrets and settings go in before the tag** that needs them.
- **Development is shared.** Every branch push replaces what `dis-dev` runs.
- **Avoid partners' run times.** A deploy restarts DIS. HDR UK harvests at about 13:15 UK time, and ADWB imports from HDR UK at 03:00 UTC.